

Programming for Biologists: Python and R for Genomic Data Science

DATA405 Genomic Data Science • Lecture 10 • Sep 28, 2026

ATGCCGGAGGCCCCGCCTCTGCTGTTGGCAGCTGTGTTGCTGGGCCTGGTGCTGCTGGTGGTG

Jinrui Xu, Department of Biology, Howard University

Site with runnable code (nothing to install): see Canvas link • Lab 1 on Wednesday

Where we are

- **L5–L7:** the genome is a script; sequencing turns it into text; RNA-seq and ChIP-seq turn text into counts.
- **L9:** same genome, different program — cell types are gene-expression programs.
- **Today:** the two languages scientists use to read that text and those tables.
- **Tomorrow:** the probability behind DESeq2, RSEM and peak calling.
- **Wednesday:** Lab 1 — everything from today, in both languages, on real chr22 data.

Today in one sentence

A genome is a **script**. Sequencing turns it into **text**. Analysis turns text into **tables**, and tables into **claims with uncertainty**.

Python = the language of the text-and-pipeline layer.

R = the language of the table-and-statistics layer.

You will read both. You will write whichever fits the step you are on.

Question → data shape → language → verify.

What you need today

- Nothing installed. Every code line today runs on the course website inside your browser (Windows, Mac, Chromebook, iPad — identical).
- Assumed: you have seen a little Python. R is new for most of you — that is fine; we learn it by translation.
- Every code slide shows the same 3–8 lines twice: Python on the left, R on the right.
- Predict before I run. When a slide says YOUR TURN, say the answer out loud before the reveal.

Rule of the week: you must be able to explain every line you run — including lines ChatGPT wrote.

Part 1

Why two languages?

Two histories, two habits of thought — and one seam most genomics projects cross.

Python: born general-purpose (1991)

- Guido van Rossum, 1991. Designed to read like pseudocode: indentation, plain words, few symbols.
- Natural units: the string, the list, the dictionary.
- When sequencing produced text at scale (2005–2010), Python became the glue: read files, call tools, parse formats, talk to databases.
- **Genomics home:** Biopython (1999), pysam, scanpy, PyTorch.

R: born statistical (1976 → 1993)

- S language, Bell Labs, 1976 — John Chambers and colleagues. Built for statistics from day one.
- R: Ross Ihaka and Robert Gentleman, University of Auckland, 1993 — an open-source S.
- Natural units: the vector and the data frame. A t-test, a linear model and a plot are one line each.
- **Genomics home:** Bioconductor (2001) — limma, edgeR, DESeq2, Seurat. New statistical methods are usually published here first.

Side by side

	Python	R
Born	1991, general purpose	1993, statistics (from S, 1976)
Natural unit	string · list · dict	vector · data frame
Indexing starts at	0	1
Plotting	matplotlib — a library you import	plot() — part of the language
Statistics	scipy, statsmodels — libraries	t.test(), lm(), glm() — built in
Genomics packages	Biopython, pysam, scanpy, PyTorch	Bioconductor: DESeq2, edgeR, limma, Seurat
Where you meet it	pipelines, machine learning, web	statistics, figures for the paper

Neither is “the real one.” Pipelines end in a table; the table is where R begins.

The same plot, two cultures (CADSA workshop)

In R, plotting is not an add-on — it is part of the core language.

Python

```
import matplotlib.pyplot as plt

plt.scatter(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

R

```
plot(x, y)
```

For a biologist starting out, R is simpler for plotting and testing; Python is simpler for files and text.

A real genomics project crosses the seam

- **Reads (FASTQ) → mapping → counts:** shell + Python (or Snakemake/Nextflow pipelines).
- **Counts → normalization → tests → figures:** R (DESeq2, base graphics).
- **Single-cell at scale, deep learning, images:** Python (scanpy, PyTorch).
- Most labs keep both. Reading both lets you read ~90 % of the code on GitHub and Bioconductor.

Learn one language well and the other by translation. Today we translate.

Part 2

Same idea, two syntaxes

Variables, text, loops, functions — the differences are mostly surface.

A variable, a number, a print

Python

```
gene = "COMT"  
n = 271  
print(gene, "has", n, "amino acids")
```

R

```
gene <- "COMT"  
n <- 271  
cat(gene, "has", n, "amino acids\n")
```

Python: = R: <- (R accepts = too). print vs cat: cat needs its own newline \n.

Text: length and letters

Python

```
dna = "ATGCCGGAGGCC"  
len(dna)           # 12  
dna[0]             # 'A'   ← index 0  
dna[3:6]           # 'CCG' letters 4,5,6  
dna[-1]            # 'C'   ← last
```

R

```
dna <- "ATGCCGGAGGCC"  
nchar(dna)         # 12  
substr(dna, 1, 1)   # "A"   ← position 1  
substr(dna, 4, 6)   # "CCG"  
substr(dna, 12, 12) # "C"
```

Python counts from 0 and stops before the end index; R counts from 1 and includes the end.

0 or 1?

YOUR TURN Both cells refer to the same three letters. Which Python slice equals R's `substr(dna, 4, 6)`?

- A. `dna[4:6]`
- B. `dna[3:6]`
- C. `dna[3:5]`
- D. `dna[4:7]`

ANSWER B — Python starts at 0 and stops before 6: positions 3, 4, 5 = letters 4, 5, 6. This is the single most common bug in genomics code.

Counting letters: the first real difference

Python

```
dna = "ATGCCGGAGGCC"
dna.count("G")           # 5

g = dna.count("G")
c = dna.count("C")
gc = (g + c) / len(dna)  # 0.75
```

R

```
dna <- "ATGCCGGAGGCC"
bases <- strsplit(dna, "")[[1]] # letters
bases           # "A" "T" "G" ...
sum(bases == "G") # 5

gc <- mean(bases %in% c("G", "C")) # 0.75
```

Python works on one string. R turns the string into a vector, then every question is a vector question (sum, mean, %in%).

Loops

Python

```
for base in "ATG":  
    print(base)  
  
for i in range(3):  
    print(i)           # 0 1 2
```

R

```
for (base in c("A", "T", "G")) {  
    cat(base, "\n")  
}  
  
for (i in 1:3) {  
    cat(i, "\n")       # 1 2 3  
}
```

Python: a colon and indentation. R: parentheses and braces. In R you rarely need the loop — the vector does it.

If / else

Python

```
q = 35
if q >= 30:
    print("good")
elif q >= 20:
    print("ok")
else:
    print("bad")
```

R

```
q <- 35
if (q >= 30) {
    cat("good\n")
} else if (q >= 20) {
    cat("ok\n")
} else {
    cat("bad\n")
}
```

Same logic, different punctuation. Read the condition, ignore the brackets.

Functions

Python

```
def gc_content(dna):  
    g = dna.count("G")  
    c = dna.count("C")  
    return (g + c) / len(dna)  
  
gc_content("ATGCCGGAGG")    # 0.7
```

R

```
gc_content <- function(dna) {  
    bases <- strsplit(dna, "")[[1]]  
    mean(bases %in% c("G", "C"))  
}  
  
gc_content("ATGCCGGAGG")    # 0.7
```

A function is a named recipe. Python says return; R returns the last value.

Collections: list vs vector, dict vs named vector

Python

```
lengths = [816, 1260, 28204]
lengths[0]          # 816
sum(lengths) / len(lengths)
[x / 1000 for x in lengths]

code = {"ATG": "M", "GTG": "V"}
code["GTG"]         # 'V'
```

R

```
lengths <- c(816, 1260, 28204)
lengths[1]          # 816
mean(lengths)
lengths / 1000       # element-wise

code <- c(ATG = "M", GTG = "V")
code["GTG"]         # "V"
```

Python list $\approx$ R vector. Python dict $\approx$ R named vector (or list). Arithmetic on a whole vector is R's superpower.

The four differences that actually matter

	Python	R	Why you care
Index of first item	0	1	BED is 0-based; GTF/VCF are 1-based — every coordinate bug
Unit of thought	one string / one item	a whole vector	R needs fewer loops; Python is easier to trace step by step
Assignment	=	<-	just habit
Blocks	indentation	{ }	copy-paste indentation errors in Python

Everything else is vocabulary. The Rosetta-stone page on the site has the 60 most common translations.

Part 3

Biology as strings

COMT: read a FASTA, count, reverse-complement, translate, mutate one letter.

Our gene today: COMT (chr22)

- **Catechol-O-methyltransferase** — breaks down dopamine, epinephrine, norepinephrine in the prefrontal cortex.
- chr22:19,941,772–19,969,975 (GRCh38), + strand, 6 exons; the canonical MB-COMT protein is 271 amino acids.
- **rs4680 (Val158Met)**: one G→A in codon 158 changes valine to methionine and lowers enzyme activity 3–4 fold. Allele frequency ≈ 0.5 in Europeans.
- Same chromosome as your BIOL419 RNA-seq lab; same reference files. Nothing here is synthetic.

Reading a FASTA file

A FASTA file: one header line starting with >, then the sequence over many lines.

Python

```
def read_fasta(path):
    seq = []
    for line in open(path):
        if not line.startswith(">"):
            seq.append(line.strip())
    return "".join(seq)

cds = read_fasta("data/COMT_cds.fa")
len(cds)          # 816
```

R

```
read_fasta <- function(path) {
  lines <- readLines(path)
  keep <- !startsWith(lines, ">")
  paste(lines[keep], collapse = "")
}

cds <- read_fasta("data/COMT_cds.fa")
nchar(cds)          # 816
```

Python: loop line by line, skip headers, join. R: read all lines, keep the non-headers, paste.

GC content of COMT

Python

```
g = cds.count("G")
c = cds.count("C")
gc = (g + c) / len(cds)
print(round(gc * 100, 1), "%")    # 61.8 %
```

R

```
bases <- strsplit(cds, "")[[1]]
gc <- mean(bases %in% c("G", "C"))
cat(round(gc * 100, 1), "%\n")    # 61.8 %
```

61.8 % — the human genome averages 41 %. COMT is a GC-rich gene on a GC-rich chromosome (why that matters for sequencing: L6).

Reverse complement

Reads come from either strand. Reverse the string, then complement every letter.

Python

```
complement = {"A": "T", "T": "A",  
              "G": "C", "C": "G"}  
rc = "".join(complement[b]  
              for b in reversed(cds))  
rc[:20]      # 'TCAGGGCCCTGCTTCGCTGC'
```

R

```
complement <- c(A = "T", T = "A",  
                G = "C", C = "G")  
rc <- paste(rev(complement[bases]),  
            collapse = "")  
substr(rc, 1, 20) # "TCAGGGCCCTGCTTCGCTGC"
```

Look-up table + reverse. Note R looks up all letters at once: `complement[bases]`.

Predict

YOUR TURN The CDS ends with the stop codon TGA. What are the first three letters of its reverse complement?

- A. ACT
- B. TCA
- C. AGT
- D. TGA

ANSWER B — reverse TGA → AGT, then complement → TCA. The output above starts with TCA. Verify with a case you already know before trusting the code on new data.

The genetic code is a look-up table

Python

```
bases = "TCAG"
aas = "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR" \
      "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"
codon_table = {}
i = 0
for a in bases:
    for b in bases:
        for c in bases:
            codon_table[a + b + c] = aas[i]
            i += 1
codon_table["ATG"] # 'M'
```

R

```
b <- c("T", "C", "A", "G")
aas <- strsplit(paste0(
  "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR",
  "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"), "")[[1]]
codons <- character(0)
for (x in b) for (y in b) for (z in b)
  codons <- c(codons, paste0(x, y, z))
codon_table <- setNames(aas, codons)
codon_table["ATG"] # "M"
```

64 codons → 21 outcomes. Three nested loops in TCAG order build the table; the 64-letter string is the standard code.

Translate

Python

```
def translate(dna):
    protein = ""
    for i in range(0, len(dna) - 2, 3):
        protein += codon_table[dna[i:i+3]]
    return protein

protein = translate(cds)
protein[:10]          # 'MPEAPPLLLA'
len(protein) - 1      # 271 aa (+ stop)
```

R

```
translate <- function(dna) {
  n <- nchar(dna) %/% 3
  starts <- seq(1, by = 3, length.out = n)
  cod <- substring(dna, starts, starts + 2)
  paste(codon_table[cod], collapse = "")
}

protein <- translate(cds)
substr(protein, 1, 10)  # "MPEAPPLLLA"
nchar(protein) - 1     # 271
```

Python walks the string three letters at a time. R computes all codon start positions, then looks them all up at once.

One letter, one amino acid: rs4680

Codon 158 is GTG (valine). rs4680 changes its first G to A.

Python

```
i = (158 - 1) * 3          # 471
cds[i:i+3]                 # 'GTG' → V

mutant = cds[:i] + "A" + cds[i+1:]
mutant[i:i+3]              # 'ATG' → M
translate(mutant)[157]     # 'M'
```

R

```
i <- (158 - 1) * 3 + 1      # 472
substr(cds, i, i + 2)       # "GTG" → V

mutant <- cds
substr(mutant, i, i) <- "A"
substr(mutant, i, i + 2)    # "ATG" → M
substr(translate(mutant), 158, 158) # "M"
```

Val158Met in six lines. Python offset 471 and R position 472 are the SAME letter — the off-by-one lives in your head, not the genome.

Substitution vs deletion

YOUR TURN Instead of changing the G to A, delete it. What happens to the protein?

- A. One amino acid changes, as before
- B. Every amino acid after 158 changes and a stop appears early
- C. Nothing — DNA repair fixes it
- D. The protein gets one amino acid shorter

ANSWER B — a frameshift: every codon downstream is read in the wrong frame. This is why indels in coding sequence are usually far worse than substitutions (L5 sickle-cell logic, generalized).

Trap: coordinates

- **GTF, VCF, SAM:** 1-based, both ends included.
- **BED, BAM (internally), Python slices:** 0-based, end excluded.
- rs4680 is chr22:19,963,748 in GRCh38 — and 19,951,271 in GRCh37. Same SNP, different build, 12,477 bp apart.
- A correct program on the wrong coordinate system or the wrong build is still wrong — and it will not crash.

Before trusting any position: which base is 0 and which build is it?

Part 4

Biology as tables

Once reads are counted, everything is a table: read, filter, sort, group, summarize, join.

Two tables from chromosome 22

- **chr22_genes.tsv** — 1,747 GENCODE genes: gene_id, gene_name, gene_type, start, end, strand, length_bp.
- **chr22_expression.tsv** — RSEM output for one ENCODE RNA-seq sample (100,000 reads, your BIOL419 Lab 4 pipeline): expected_count and TPM per gene.
- Python's table is the pandas DataFrame. R's table is the built-in data.frame.
- **Five verbs cover most analysis:** read · filter · sort · group + summarize · join.

If you can do the five verbs in both languages, you can read most analysis scripts.

Read and look

Python

```
import pandas as pd
genes = pd.read_csv("data/chr22_genes.tsv",
                    sep="\t")
genes.shape                # (1747, 7)
genes.head(3)
genes["gene_type"].value_counts().head(3)
```

R

```
genes <- read.delim("data/chr22_genes.tsv")
dim(genes)                # 1747 7
head(genes, 3)
head(sort(table(genes$gene_type),
              decreasing = TRUE), 3)
```

lncRNA 747 • protein_coding 447 • processed_pseudogene 142. Column access: `df["x"]` in pandas, `df$x` in R.

Filter and sort

Python

```
pc = genes[genes["gene_type"] ==  
             "protein_coding"]  
len(pc)                # 447  
pc["length_bp"].median() # 23,780  
  
genes.sort_values("length_bp",  
                  ascending=False).head(3)
```

R

```
pc <- genes[genes$gene_type ==  
            "protein_coding", ]  
nrow(pc)                # 447  
median(pc$length_bp)    # 23,780  
  
head(genes[order(-genes$length_bp), ], 3)
```

Filter = a TRUE/FALSE mask inside square brackets, in both languages. R needs the comma: rows, columns.

Group and summarize

Python

```
genes.groupby("gene_type")["length_bp"] \
    .median() \
    .sort_values(ascending=False) \
    .head(3)

# protein_coding      23780
# lncRNA               4781
# transcribed_unprocessed_pseudogene 4039
```

R

```
head(sort(tapply(genes$length_bp,
                 genes$gene_type, median),
      decreasing = TRUE), 3)

# or: aggregate(length_bp ~ gene_type,
#               data = genes, FUN = median)
```

groupby → tapply / aggregate. Protein-coding genes are 5× longer than lncRNAs on chr22: introns.

Join two tables

Python

```
expr = pd.read_csv(
    "data/chr22_expression.tsv", sep="\t")
m = genes.merge(expr[["gene_id", "TPM"]],
                on="gene_id")
frac_on = m.groupby("gene_type")["TPM"] \
    .apply(lambda s: (s > 1).mean())
frac_on.sort_values(ascending=False).head(3)
```

R

```
expr <- read.delim(
    "data/chr22_expression.tsv")
m <- merge(genes,
           expr[, c("gene_id", "TPM")],
           by = "gene_id")
frac_on <- tapply(m$TPM > 1,
                  m$gene_type, mean)
head(sort(frac_on, decreasing = TRUE), 3)
```

merge on a shared key. Result: ~47 % of protein-coding genes are ON (TPM > 1) in this sample; most lncRNAs are OFF.

Same genome, different program — from a table

YOUR TURN The five highest-TPM genes are IGLC2, IGLV2-23, ACTBP15, ENSG00000241838, HMGN2P9. What was sequenced, and why are two of the five pseudogenes?

- IGL* = immunoglobulin lambda light chain → a B-cell-derived sample (L9: cell type = expression program)
- ACTBP15 is a processed pseudogene of β -actin. Its reads are almost certainly β -actin mRNA mis-mapped to the chr22 copy — because only chr22 was in the reference (L6 mapping; tomorrow: EM).

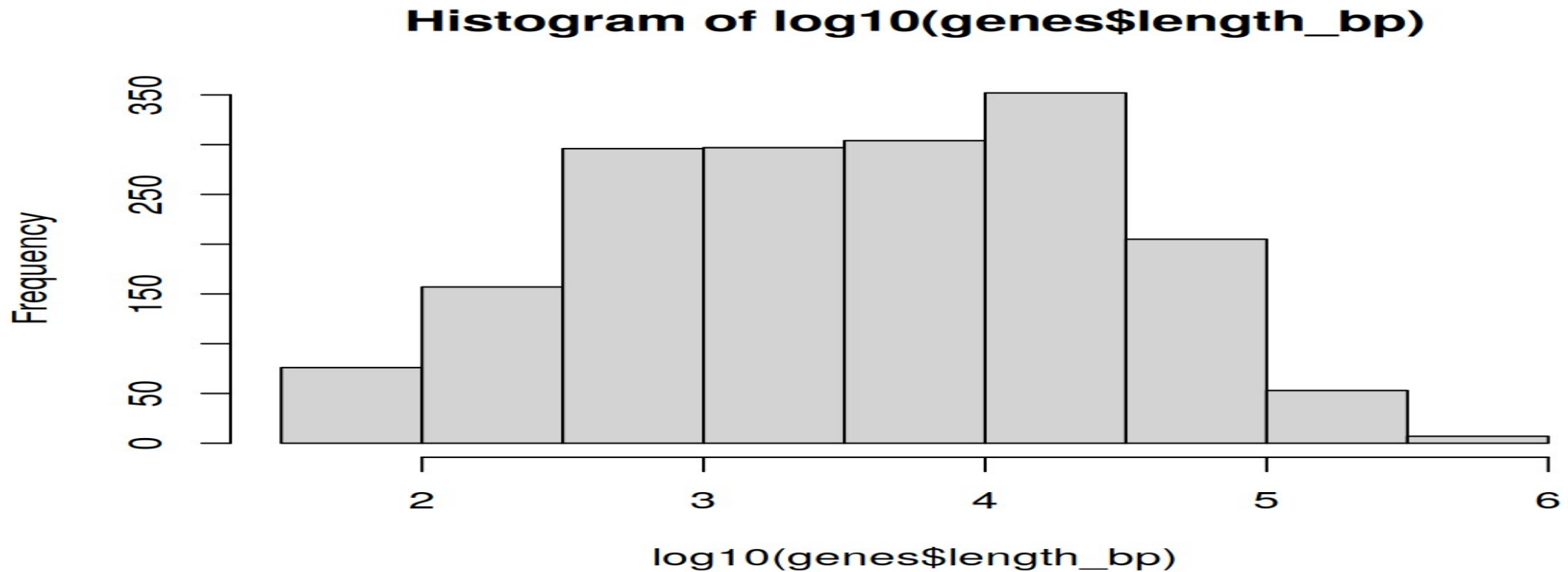
ANSWER A table can answer a biology question — and also expose a pipeline artefact. Both are data science.

Part 5

Plots and a first test

Where R pulls ahead — and how to make a figure that survives the journal's shrink.

Default R plot: fine on screen, unreadable in a paper



`hist(log10(genes$length_bp))` with defaults, then shrunk to a single column

CADSA workshop lesson: what you see in R is not what you get in the manuscript.

Think in inches first (CADSA recipe)

Python

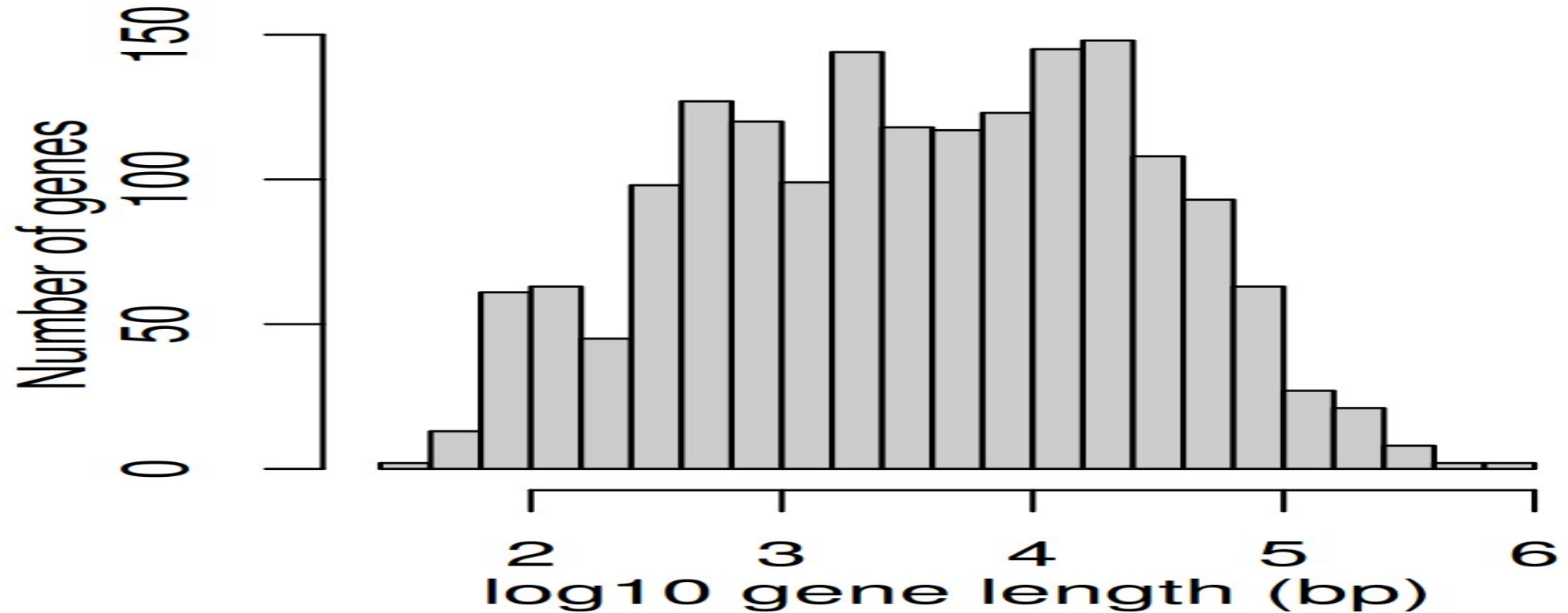
```
import matplotlib.pyplot as plt, numpy as np
fig, ax = plt.subplots(figsize=(3.2, 3.2))
ax.hist(np.log10(genes["length_bp"]),
        bins=30, color="grey")
ax.set_xlabel("log10 gene length (bp)")
ax.set_ylabel("Number of genes")
for s in ["top", "right"]:
    ax.spines[s].set_visible(False)
plt.tight_layout(); plt.show()
```

R

```
png("fig1.png", width = 3.2, height = 3.2,
     units = "in", res = 300)
par(mar = c(4, 4, 1, 1))
hist(log10(genes$length_bp), breaks = 30,
     col = "grey80", main = "", xlab = "",
     ylab = "", axes = FALSE)
axis(1, line = 0.1); axis(2, line = 0.1)
title(xlab = "log10 gene length (bp)",
      line = 2)
title(ylab = "Number of genes", line = 2)
dev.off()
```

Canvas 3.2 in (one journal column) → margins → draw nothing automatic → add axes and labels yourself.

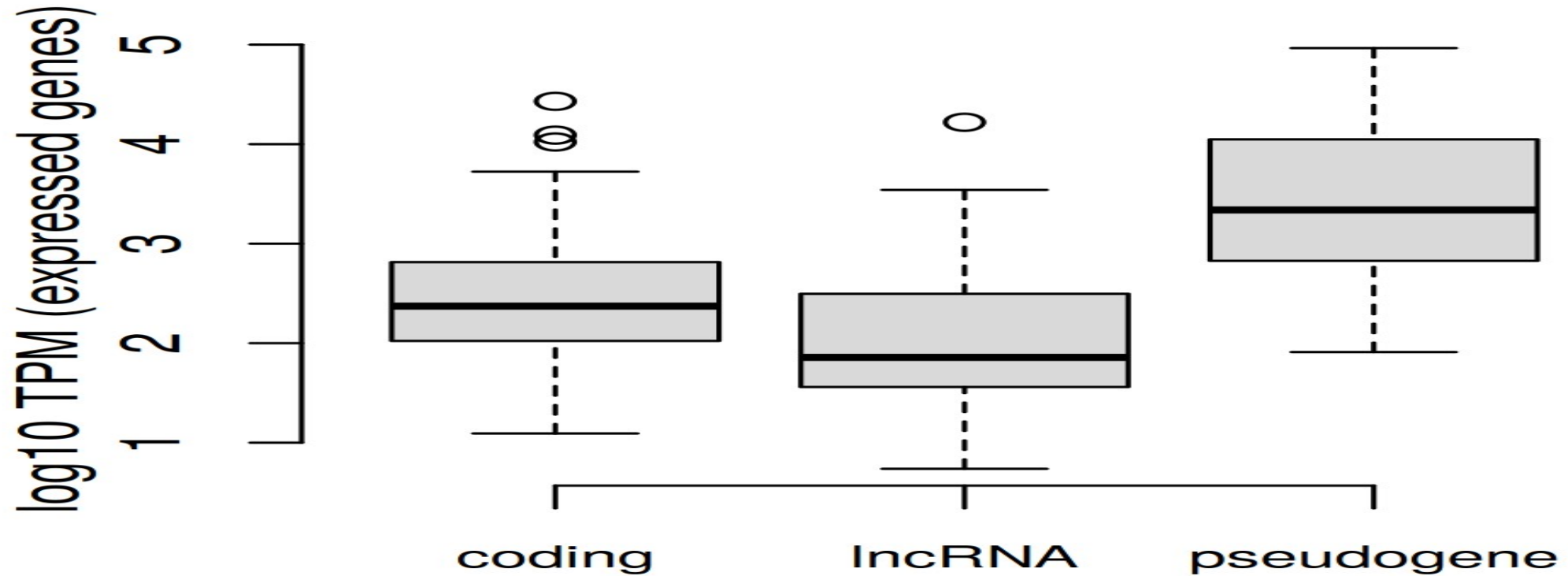
Result: readable at journal size



3.2 × 3.2 in, base R. Most chr22 genes are 1–100 kb; a few exceed 500 kb (LARGE1, TTC28, SYN3).

Same data as two slides ago. Only the recipe changed.

Expressed genes by gene type (base R boxplot)



```
boxplot(log10(TPM) ~ gene_type, data = m)
```

A formula ($y \sim \text{group}$) is R's native way to say 'y by group' — the same syntax you will use in DESeq2's design tomorrow.

Airway: a real experiment with replicates

- Himes et al. 2014 (PLOS ONE): human airway smooth-muscle cells from 4 donors.
- Each donor's cells untreated and treated with dexamethasone (an asthma steroid) → 8 RNA-seq samples.
- The teaching dataset of the DESeq2 tutorial. Here: 19,271 genes × 8 samples of raw counts.
- **Question:** is gene ENSG00000152583 (SPARCL1) changed by the drug?

One gene: boxplot and t-test

Python

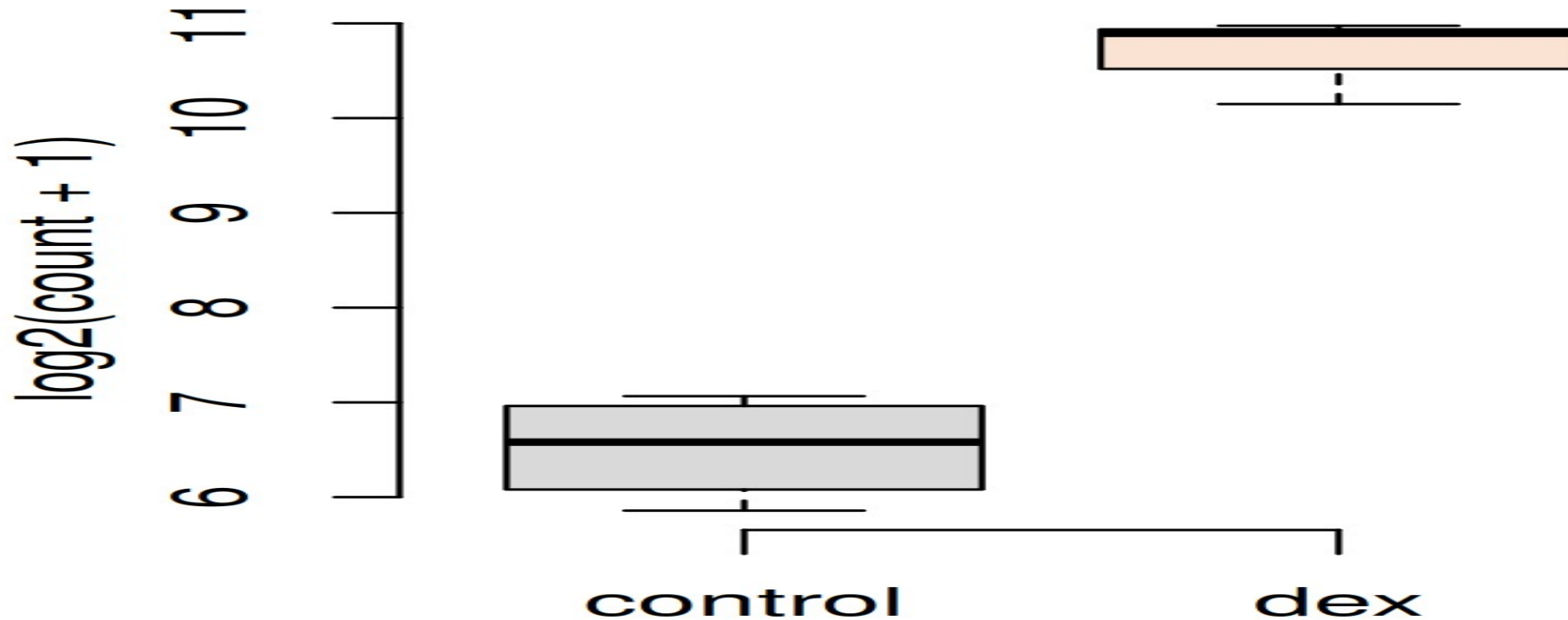
```
from scipy import stats
cts = pd.read_csv("data/airway_counts.tsv",
                  sep="\t", index_col=0)
meta = pd.read_csv(
    "data/airway_metadata.tsv", sep="\t")
logc = np.log2(cts + 1)
ctrl = meta["id"][meta["dex"] == "control"]
trt = meta["id"][meta["dex"] == "treated"]
g = "ENSG00000152583"
stats.ttest_ind(logc.loc[g, trt],
                 logc.loc[g, ctrl])
# statistic=12.5, pvalue=1.6e-05
```

R

```
cts <- read.delim("data/airway_counts.tsv",
                  row.names = 1)
meta <- read.delim(
    "data/airway_metadata.tsv")
logc <- log2(cts + 1)
ctrl <- meta$id[meta$dex == "control"]
trt <- meta$id[meta$dex == "treated"]
g <- "ENSG00000152583"
a <- unlist(logc[g, ctrl])
b <- unlist(logc[g, trt])
boxplot(list(control = a, dex = b))
t.test(b, a)
# t = 12.5, p = 3e-05, 95% CI of difference...
```

R prints the whole test — CI, means, method — with one call. That is what 'statistics is first-class' means.

SPARCL1: log2 counts, control vs dexamethasone



About 16× more reads after treatment ($\log_2$ difference ≈ 4)

Convincing for one gene. Tomorrow: why doing this 19,271 times, unpaired, on counts, is the wrong tool.

Why is this t-test suspicious?

YOUR TURN We have 19,271 genes and 4 donors. Name two things the plain t-test ignores.

- It ignores that the 8 samples are 4 pairs (same donor, $\pm$ drug).
- It ignores that counts are not normal — variance grows with the mean.
- It ignores that we run 19,271 tests: at $p < 0.05$ we expect ~ 960 false positives by chance.

ANSWER All three. Lecture 11 gives you the negative binomial, the paired design and Benjamini-Hochberg.

Part 6

Reads: the text sequencing produces

FASTQ, Phred quality, and why R thinks in matrices.

A FASTQ record is four lines

@HISEQ:476:HLGH2BCXX:2:2105:4102:27370 1:N:0:TATCCTCT

CAGTAACAAACAAGATTTCACCACTGAATATTGTGATGTG...

+

ADDDDIHHIIHHIIHHIIHHIIEHIIIIEE...

Quality letter → number: $Q = \text{ASCII code} - 33$. 'I' = $73 - 33 = 40$. $P(\text{error}) = 10^{(-Q/10)}$: $Q_{40} = 1$ in 10,000.

reads_5k.fastq: 5,000 real 100-nt reads (ENCODE, HiSeq), randomly drawn from your BIOL419 Lab 4 file.

Parse the file

Python

```
seqs, quals = [], []
with open("data/reads_5k.fastq") as f:
    while True:
        name = f.readline()
        if not name:
            break
        seqs.append(f.readline().strip())
        f.readline()           # '+'
        quals.append(f.readline().strip())
len(seqs)                     # 5000
```

R

```
lines <- readLines("data/reads_5k.fastq")
n <- length(lines)
seqs <- lines[seq(2, n, by = 4)]
quals <- lines[seq(4, n, by = 4)]
length(seqs)                  # 5000
```

Python reads four lines at a time. R reads everything, then takes every 4th line starting at 2 and at 4.

Decode quality; mean per position

Python

```
q = [ord(c) - 33 for c in quals[0]]
q[:5]          # [35, 35, 35, 35, 35]

L = len(seqs[0])
mean_q = []
for i in range(L):
    tot = sum(ord(s[i]) - 33 for s in quals)
    mean_q.append(tot / len(quals))
```

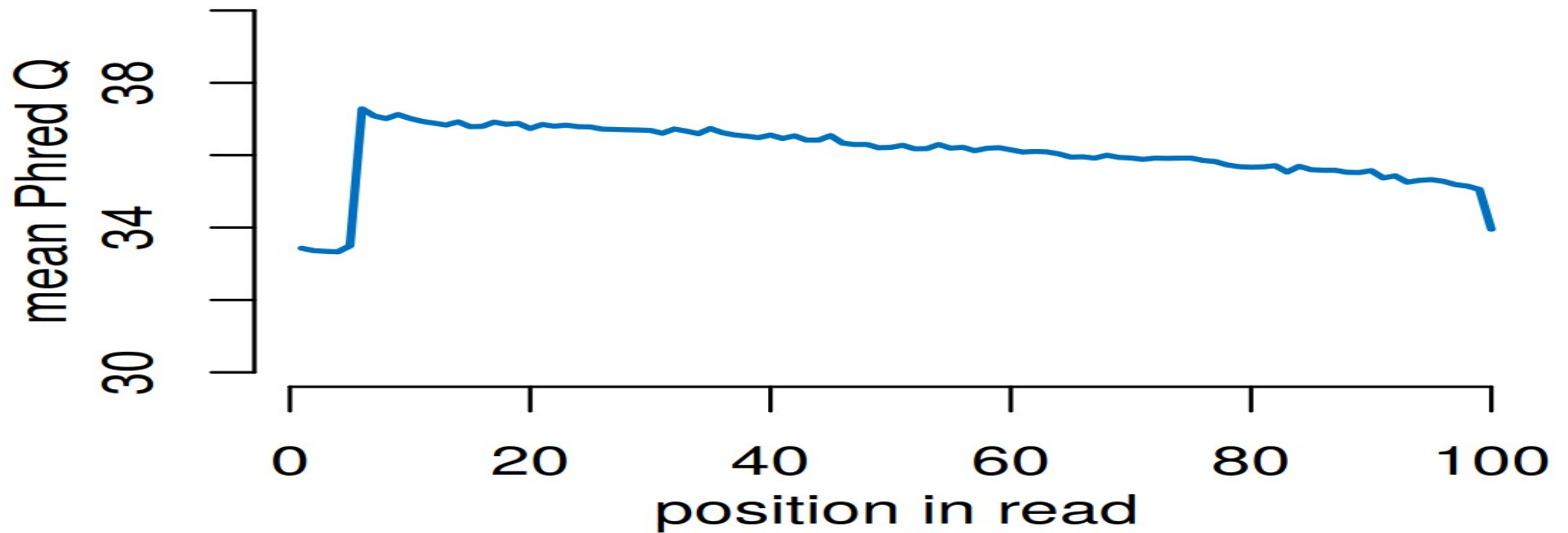
R

```
q <- utf8ToInt(quals[1]) - 33
q[1:5]          # 35 35 35 35 35

Q <- t(sapply(quals,
              function(s) utf8ToInt(s) - 33))
dim(Q)          # 5000 100
mean_q <- colMeans(Q)
```

Python loops position by position. R builds a $5,000 \times 100$ matrix and takes column means. Same numbers; different way of thinking.

Mean quality along the read



Q $\approx$ 33 at position 1, $\approx$ 37 in the middle, drifting to $\approx$ 35 by position 100

This is the plot every QC tool (FastQC) shows first. You just made it from raw bytes.

Count reads by content

Python

```
sum("N" in s for s in seqs)          # 6
sum("A" * 15 in s for s in seqs)     # 63
gc = [(s.count("G") + s.count("C")) / len(s)
       for s in seqs]
sum(gc) / len(gc)                    # 0.474
```

R

```
sum(grepl("N", seqs))                # 6
sum(grepl(strrep("A", 15), seqs))     # 63
gc <- sapply(strsplit(seqs, ""),
             function(s) mean(s %in% c("G", "C"))
             )
mean(gc)                              # 0.474
```

Reads are 47 % GC; the genome is 41 %. RNA-seq samples exons, and exons are GC-rich than introns and intergenic DNA.

Part 7

Working like a scientist

Packages, notebooks, reproducibility, and using ChatGPT without being used by it.

Ecosystems

	Python	R
Install packages	pip install biopython · conda install	install.packages("x") · BiocManager::install("DESeq2")
Package home	PyPI · conda-forge · bioconda	CRAN · Bioconductor
Load	import pandas as pd	library(DESeq2)
Notebook	Jupyter (.ipynb)	R Markdown / Quarto (.Rmd, .qmd)
IDE	VS Code, JupyterLab	RStudio
Environments	conda env, venv	renv
Record versions	pip freeze	sessionInfo()

Bioconductor is the reason R stays essential: DESeq2, edgeR, limma live there and nowhere else.

The notebook is the lab notebook

- Code, output, and your comments in one file — the analysis explains itself six months later.
- The course website is a Quarto notebook rendered to HTML; the code you run in the browser is the same code you would run locally.
- **Reproducibility habits:** never edit raw data by hand • set random seeds • keep the script that made every figure • record package versions.
- When a project switches languages, the seam is a file: Python writes a TSV, R reads it. Keep the seam explicit.

If you cannot re-run it, it is not a result yet.

ChatGPT writes code. You verify it.

- TRACE prompting, as in your proposal: Task, Role, Audience, Context, Example.
- **Then test on a case whose answer you already know:** COMT codon 158 must be GTG; the protein must start with M and have 271 aa; the reverse complement of TGA must be TCA.
- Ask it to explain every line. If you cannot explain a line, do not run it on real data.
- Copilot-style autocomplete is off in this lab: the exercises are exactly the lines it would type for you.

Lab 1 ends with a ChatGPT function that fails two of three tests. Your job will be to find out why.

The five bugs that bit real papers

Bug	What happens	Defence
0- vs 1-based	Off by one base: wrong codon, wrong variant	Know which base is 0 for every file format
Strand	A minus-strand k-mer never matches	Reverse-complement before searching
Reference build	GRCh37 positions in a GRCh38 file	Write the build in every file name
Excel gene names	SEPT7 → 7-Sep, MARCH1 → 1-Mar (≈20 % of supplementary tables; genes renamed in 2020)	Never open a gene list in Excel; use read.delim / pandas
Silent type coercion	A numeric column read as text; p-values sorted as strings	Check dtypes / str(df) after every read

None of these crash. All of them produce a confident, wrong table.

Part 8

The mental model

Question → data shape → language → verify.

Which language for which step?

The data are...	Think...	Reach for...
Sequences, reads, files, a tool to glue	text	Python (or the shell)
One row per gene, columns of numbers	table	either; R if statistics come next
Counts with replicates and a hypothesis	table + uncertainty	R: DESeq2, edgeR, limma
Images, deep learning, single cell at scale	arrays	Python: PyTorch, scanpy
A figure for the paper	inches	base R (or matplotlib) with the CADSA recipe

What you can now do

- Read a FASTA/FASTQ and compute GC, reverse complement, quality — in both languages.
- Translate DNA and predict the effect of a single-letter change (rs4680).
- Read, filter, sort, group and join gene tables — in both languages.
- Make a figure that survives the journal's shrink.
- Run a t-test — and say why it is not enough (tomorrow).

Wednesday's lab does all of this on the same files. Come with a laptop and a browser; nothing else.

Lab 1 on Wednesday (1.5 h)

- Ex 1 Hello, two languages • Ex 2 COMT string operations + rs4680 • Ex 3 FASTQ quality
- Ex 4 Tables: genes, expression, mismatches (which look like real variants?)
- Ex 5 The t-test and the 19,271-tests trap • Ex 6 Catch ChatGPT's bug
- Everything runs on the course website; the handout has the same exercises with blanks.
- Lab report: answers to Q1–Q7, the fixed function in both languages, one 3.2-inch figure.

Questions?

jinrui.xu@howard.edu

Course site: runnable notes, Rosetta stone, Lab 1, data provenance, local setup for Windows and Mac.